Handling Missing Categorical Data: Most Frequent Value and Missing Category

Contents

1. Overview . . . 2
2. Most frequent value imputation . . . 3
   - 2.1 The idea . . . 3
   - 2.2 When to use it . . . 4
   - 2.3 Advantages and disadvantages . . . 4
3. Missing category imputation . . . 5
   - 3.1 The idea . . . 5
   - 3.2 When to use it . . . 5
   - 3.3 Advantages and disadvantages . . . 6
4. The house price data . . . 6
5. Mode imputation on real data . . . 7
   - 5.1 Split first . . . 8
   - 5.2 `GarageQual`: one category dominates . . . 8
   - 5.3 `FireplaceQu`: two categories nearly tied . . . 10
   - 5.4 Mode imputation with `SimpleImputer` . . . 11
6. Missing category imputation on real data . . . 12
7. Choosing between the two . . . 13
8. Summary . . . 15
9. Sources . . . 15
10. Key terms . . . 15

Where this fits: Pipeline map step 4 (Clean). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	**4 Clean**	5 Features	6 Reduce
7 Split	8 Model	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Missing values; Simple imputation (mean, median, mode, constant).
- **Compare with:** Complete case analysis; Random sample imputation; KNN imputer; Outliers.

1. Overview

Key point: A gap in a categorical feature is filled either with the most frequent category (**mode** (G-1251) imputation) or with a new category called "Missing".

Where each technique is covered mapped the techniques for missing data, and mean and median imputation filled gaps (missing values) in numerical features. A **feature** (G-772) is an input variable (one column of the data table), the **target** (G-1949) is the output we predict, and an **observation** (G-1374) is one record (one row). A categorical feature (one whose values are labels, such as a city name) has no mean or median, so it needs its own techniques. This Note covers the two main ones.

Figure 1 shows both on a small `city` column with two gaps, marked NaN ("not a number", the mark pandas uses for a missing value):

- **Mode imputation** fills every gap with the most frequent city, Mumbai. The column keeps its three categories.
- **Missing category imputation** (G-1232) fills every gap with the word "Missing". The column now has four categories.

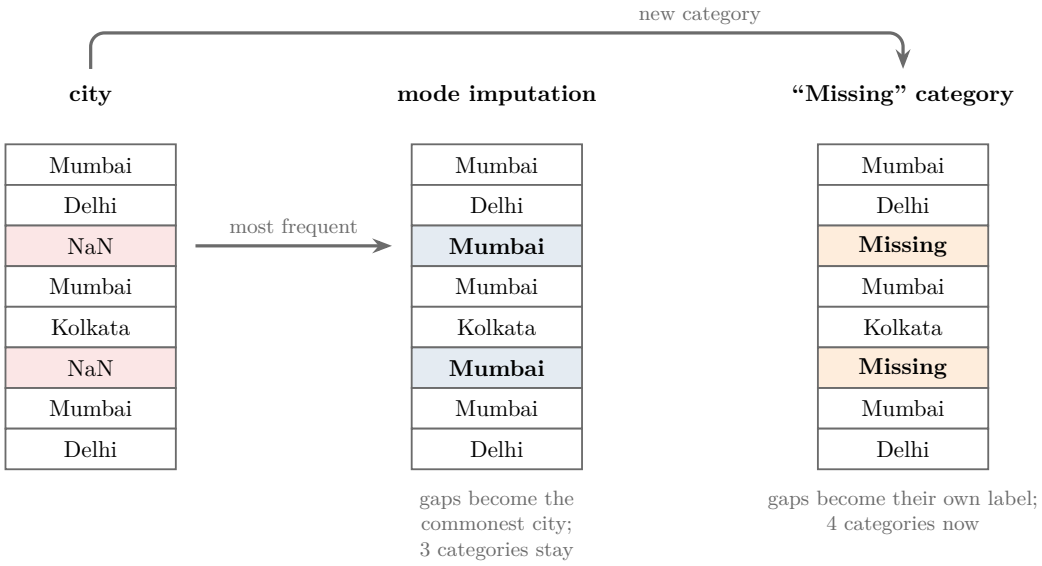


Figure 1: Two ways to fill gaps in a categorical column: the most frequent category, or a new category “Missing”

Both are **univariate imputation** (G-2051) techniques: they look only at the feature with the gap. scikit-learn’s `SimpleImputer` (G-1809) does both.

> **Extra:** A third technique, filling each gap with a random value drawn from the column, works for numerical and categorical columns alike. **Random sample imputation** (G-1616) covers it.

2. Most frequent value imputation

> **Key point: Mode** (G-1251) imputation replaces every missing value with the category that appears most often in the column.

2.1 The idea

> **Key point:** The mode of a column is its most frequent value; every gap gets that value.

The **mode** (G-1251) of a column is the value that occurs most often in it. **Most frequent value imputation**, or **mode imputation** (G-1264), fills every missing value of a column with its mode.

Take a column `city` with the values Mumbai, Delhi and Kolkata, plus some gaps. Mumbai appears most often, then Delhi, then Kolkata. Every gap becomes Mumbai.

Mode imputation also works on a numerical column, since numbers have a mode too. In practice the mean or median (mean and median imputation) works better there, so mode imputation is used mainly for categorical data.

2.2 When to use it

> **Key point:** Use **mode** (G-1251) imputation when the data is missing completely at random, few values are missing, and one category clearly dominates.

Mode imputation (G-1264) rests on three conditions:

1. **The data is MCAR** (G-1192) (missing completely at random, see **MCAR** (G-1192)). The gaps have no pattern, so the **observations** (G-1374) with a gap look like the others.
2. **Few values are missing**, as a rule of thumb up to about 5% of the **feature** (G-772). The 5% limit is the same as for mean and median imputation.
3. **One category dominates.** The mode should appear far more often than every other category. If Mumbai, Delhi and Kolkata each make up about a third of the column, guessing Mumbai for every gap is wrong two times out of three.

2.3 Advantages and disadvantages

> **Key point: Mode** (G-1251) imputation is quick and easy to deploy, but it inflates the most frequent category and so changes the distribution.

Advantages:

1. **Easy to apply.** One value per column, learned once.
2. **Easy to deploy.** The deployed model stores that one value and fills any gap in new data with it.

Disadvantage: it changes the distribution of the column. Every gap adds to the mode's count, so the most frequent category grows and all others shrink. The more values are missing, the bigger the change, as Section 5 shows with real numbers.

Figure 2 fills the 547 gaps of the `FireplaceQu` **feature** (G-772) in the training set (Sections 4 and 5) with its mode, Gd, a batch at a time. Watch the green bar grow past its grey outline while every other bar shrinks; the curve on the right shows Gd's share climbing from 49.1% to 72.9% as more gaps are filled.

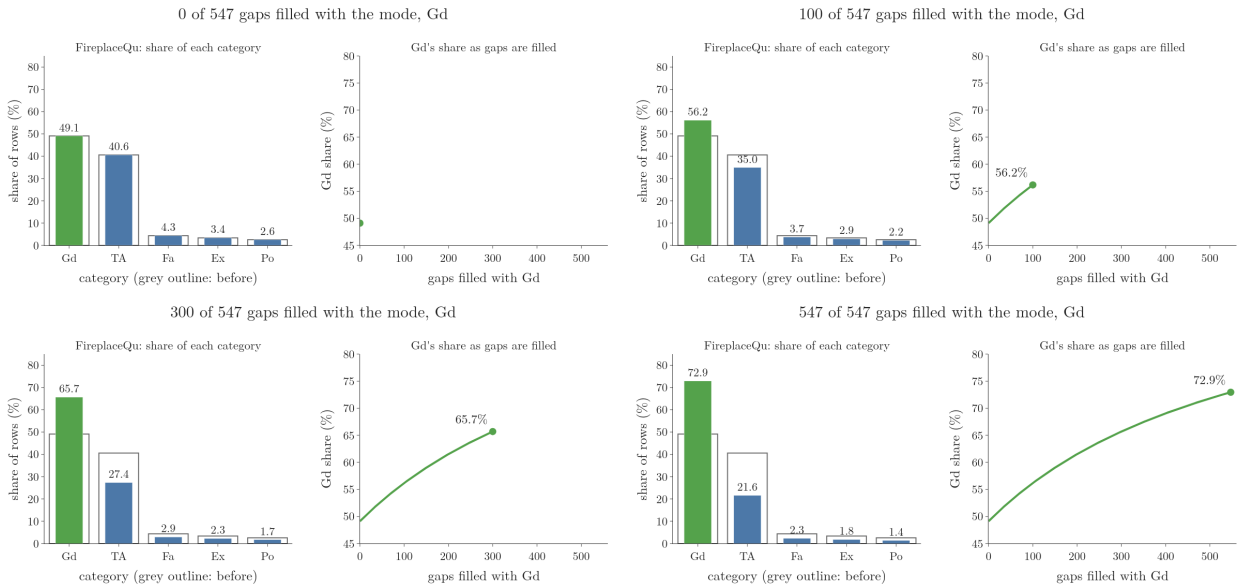


Figure 2: Mode imputation gap by gap on FireplaceQu: each filled gap adds to Gd, so Gd's share rises from 49.1% to 72.9% and every other category shrinks

3. Missing category imputation

> **Key point: Missing category imputation** (G-1232) gives every gap the same new label, "Missing", so the model can learn that the value was absent.

3.1 The idea

> **Key point:** We do not guess the missing value; we mark it as missing with a category of its own.

Missing category imputation fills every missing value of a categorical column with a new category, usually the word "Missing". The `city` column of Figure 1 then has four categories: Mumbai, Delhi, Kolkata and Missing.

The aim is to tell the ML algorithm which rows had no value. The algorithm then treats "Missing" like any other category and can learn whether it matters.

This is the categorical version of **arbitrary value imputation** (G-208) for numerical columns (**arbitrary value imputation** (G-208)), where gaps are filled with a value such as 99 or -1 that cannot occur naturally. In a categorical column, a word plays that role.

3.2 When to use it

> **Key point:** Use the "Missing" category when many values are missing or the gaps are not random.

Suppose a third or more of the `city` column is missing. Filling all of those gaps with Mumbai would make Mumbai far bigger than it really is, and the results suffer. A "Missing" category leaves the real categories as they were.

So the "Missing" category suits two cases:

- **Many values are missing**, well above 5%.
- **The data is not missing at random** (**MAR**, G-1158, or **MNAR**, G-1248; see the other two kinds). The fact that a value is missing may then carry information that the model can use.

3.3 Advantages and disadvantages

> **Key point:** The "Missing" category is easy, keeps the existing categories intact and captures why data is missing, but it fills nothing in.

Advantages:

1. **Easy to apply and deploy**, like **mode** (G-1251) imputation.
2. **Keeps the existing categories' counts** exactly as they were.
3. **Captures the missingness itself**, which helps when the gaps are not random.

Disadvantage: it does not estimate the missing value; it only labels it, so the results are not always good.

> **Extra:** After **one-hot encoding** (G-1379) (one 0/1 column per category), the "Missing" category becomes its own 0/1 column. That column is the same as a **missing indicator** (G-1233; see the missing indicator) for this **feature** (G-772).

4. The house price data

> **Key point:** We use two quality ratings from a house price dataset: one missing 5.5% of its values, the other 47.3%.

The data is the training file of Kaggle's *House Prices: Advanced Regression Techniques*: 1,460 houses sold in Ames, Iowa, with 81 columns. We use only three:

- `GarageQual`: quality of the garage.
- `FireplaceQu`: quality of the fireplaces.
- `SalePrice`: the price the house sold for, in dollars. `SalePrice` is the **target** (G-1949); since the target is a number, the task is regression (predicting a number).

Both quality columns use the same five categories:

Code	Meaning
Ex	Excellent
Gd	Good
TA	Typical/Average

Code	Meaning
Fa	Fair
Po	Poor

The two columns were chosen for their very different shares of missing values:

Column	Missing values	Share missing
`GarageQual`	81 of 1,460	5.5%
`FireplaceQu`	690 of 1,460	47.3%

Figure 3 sets the two shares against the 5% rule of thumb of Section 2.2. Watch where each bar ends: `GarageQual` sits right at the line, `FireplaceQu` far beyond it.

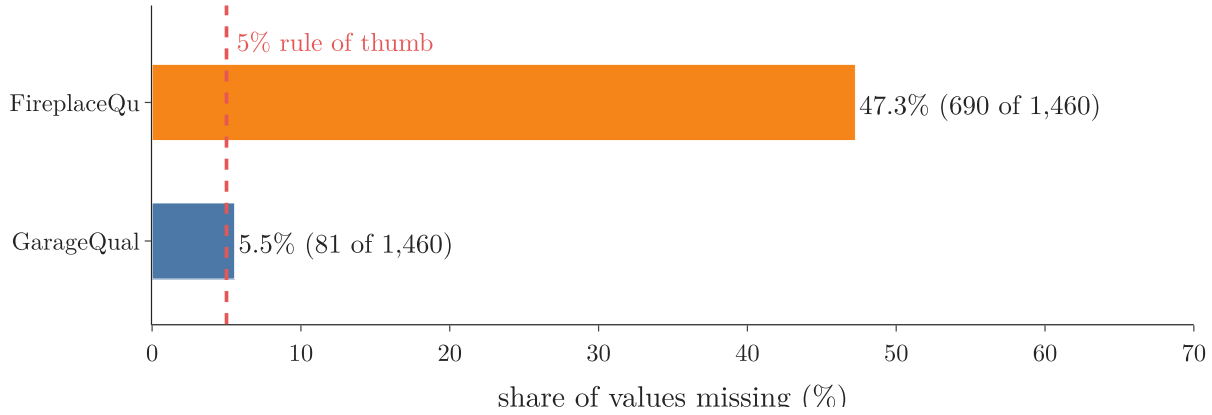


Figure 3: Share of missing values in each column, against the 5% rule of thumb for mode imputation

The file `data/house_prices.csv` keeps these three columns and two more used in Section 7. The full file, with all 81 columns, is on Kaggle.

> **Python:** Loading the columns and measuring the gaps.
> ```
> df = pd.read_csv("data/house_prices.csv")
> df.isnull().mean() * 100
> # FireplaceQu 47.26, GarageQual 5.55, SalePrice 0.0
> ```

5. Mode imputation on real data

> **Key point: Mode** (G-1251) imputation barely changes `GarageQual`, where one category dominates and few values are missing, but badly distorts `FireplaceQu`.

5.1 Split first

> **Key point:** We split the data before imputing, and learn the **mode** (G-1251) from the training set alone.

An imputer learns something from the data: here, the mode of each column. Like a scaler (scaling the inputs), it must learn only from the training set, or information from the test set leaks into training (**data leakage**, G-535). So we split first, with 80% of the houses for training.

> **Python:** Splitting before imputing.

```
X = df[["GarageQual", "FireplaceQu"]]
y = df["SalePrice"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
X_train.isnull().sum()
# GarageQual 64, FireplaceQu 547  (of 1,168 rows)
```

All numbers in the rest of this Note come from these 1,168 training houses.

5.2 `GarageQual`: one category dominates

> **Key point:** TA makes up 95% of the garages, so giving the 64 gaps the value TA hardly changes anything.

The **mode** (G-1251) of `GarageQual` is TA: 1,050 of the 1,104 known garages are rated "typical/average". Figure 4 (left) shows how strongly it dominates. Two of the three conditions of Section 2.2 hold: few gaps (5.5%) and one dominant category. The third, MCAR, is tested below with the sale prices.

The share of a category, step by step:

1. **In words:** count the rows with that category and divide by the rows that have a value.
2. **Formula:**

$$A = \text{rows with the category}$$

$$B = \text{rows with a value}$$

$$\text{share} = \frac{A}{B} \times 100 \text{ percent}$$

3. **Example:** before imputation, 1,050 of the 1,104 known garages are TA:

$$\frac{1{,}050}{1{,}104} \times 100 \text{ percent} = 95.1 \text{ percent}.$$

After imputation, the 64 gaps are TA too, and every one of the 1,168 rows has a value:

$$\frac{1{,}050 + 64}{1{,}168} = \frac{1{,}114}{1{,}168}$$

$$\frac{1{,}114}{1{,}168} \times 100 \text{ percent} = 95.4 \text{ percent}.$$

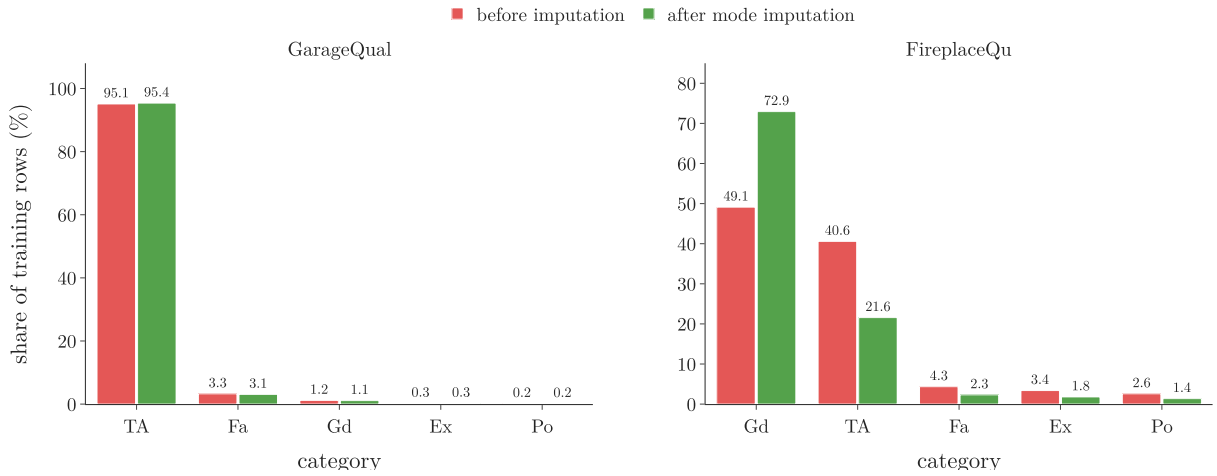


Figure 4: Category shares in the training set before and after mode imputation: GarageQual barely moves, while Gd in FireplaceQu jumps from 49.1% to 72.9%

`GarageQual`	Before	After
TA	95.1%	95.4%
Fa	3.3%	3.1%
Gd	1.2%	1.1%
Ex	0.3%	0.3%
Po	0.2%	0.2%

No share moves by more than 0.3 percentage points.

A second check looks at the **target** (G-1949). If the houses with a gap were like the TA houses, their sale prices should look alike. Figure 5 (top left) compares the two as density curves (**KDE**, G-1005; a smooth curve over a histogram, see the density plot).

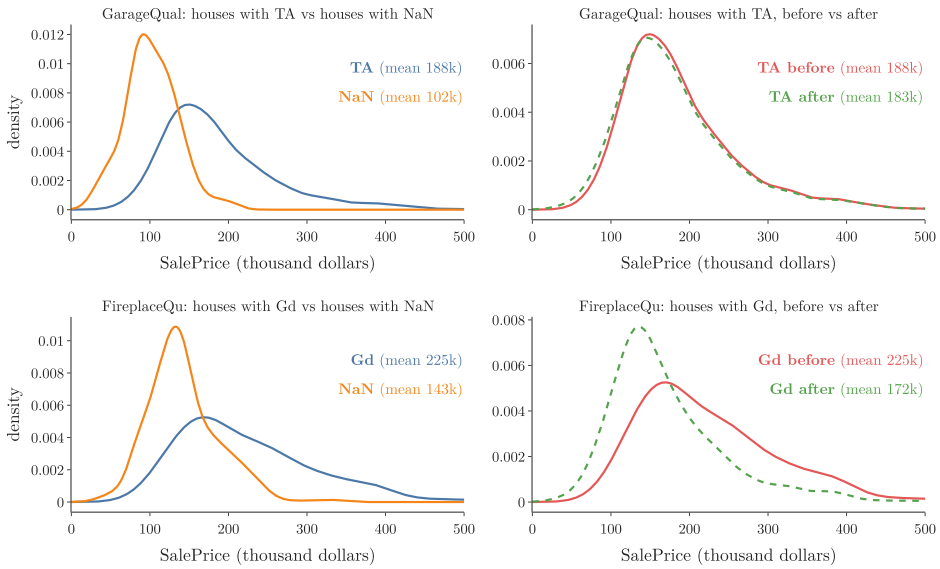


Figure 5: Sale prices as density curves. Left: houses in the most frequent category against houses with a gap. Right: the most frequent category before and after imputation

In Figure 5 the houses with a gap are labelled NaN, as in Figure 1. In the left curves, look at where each curve peaks: the curve of the houses with a gap sits well to the left of the TA curve. The means say the same in numbers: the houses with a gap sold for a mean of 102,000 dollars against 188,000 for TA houses, a difference of 86,000 dollars. Under MCAR, the gaps would not depend on any value (**MCAR** (G-1192)), so the houses with a gap would sell for about the same as the rest. They do not, so these gaps are not MCAR. Yet there are only 64 of them, so adding them to the 1,050 TA houses changes the TA curve very little (Figure 5, top right): its mean falls from 188,000 to 183,000 dollars.

For `GarageQual`, **mode imputation** (G-1264) is acceptable. Few values are missing, so even a poor guess cannot change much.

5.3 `FireplaceQu`: two categories nearly tied

> **Key point:** Gd and TA are nearly equal, and almost half the values are missing, so **mode** (G-1251) imputation turns Gd from 49% into 73% of the column.

The mode of `FireplaceQu` is Gd, but TA is close behind: 49.1% against 40.6% of the known values (Figure 4, right). No category dominates, and 46.8% of the training rows are missing. Two of the three conditions fail.

Imputing the 547 gaps with Gd changes the shares heavily:

`FireplaceQu`	Before	After
Gd	49.1%	72.9%

FireplaceQu	Before	After
TA	40.6%	21.6%
Fa	4.3%	2.3%
Ex	3.4%	1.8%
Po	2.6%	1.4%

Gd grows by the 547 imputed rows, from 305 to 852 of 1,168 rows:

$$305 + 547 = 852$$

$$852/1{,}168 = 72.9 \text{ percent}$$

Every other category loses close to half its share.

The sale prices tell the same story. Houses with a gap sold for a mean of 143,000 dollars, against 225,000 for Gd houses (Figure 5, bottom left). After imputation, 547 cheaper houses join the 305 Gd houses, and the Gd curve moves left: its mean falls from 225,000 to 172,000 dollars (Figure 5, bottom right).

For `FireplaceQu`, **mode imputation** (G-1264) is the wrong choice. The "Missing" category of Section 6 suits it better.

5.4 Mode imputation with `SimpleImputer`

> **Key point:** `SimpleImputer(strategy="most_frequent")` learns each column's **mode** (G-1251) with `fit` and fills the gaps with `transform`.

> **Python: Mode imputation** (G-1264), fitted on the training set.
>
```
from sklearn.impute import SimpleImputer

imp = SimpleImputer(strategy="most_frequent")
imp.set_output(transform="pandas")
imp.fit(X_train)          # learn the modes
imp.statistics_           # ['TA', 'Gd']
X_train = imp.transform(X_train)
X_test = imp.transform(X_test)
```
> **`statistics_`** (G-145) holds what `fit` learned: one fill value per column, in column order. `set_output(transform="pandas")` returns a DataFrame with the column names, instead of a bare array.

The same can be done in pandas with `fillna`. The mode must still come from the training set.

> **Python:** Mode imputation in pandas.
>
```
mode = X_train["GarageQual"].mode()[0]    # 'TA'
```

```
X_train["GarageQual"] = X_train["GarageQual"].fillna(mode)
X_test["GarageQual"] = X_test["GarageQual"].fillna(mode)
```
`mode()` returns every value tied for most frequent, so `[0]` takes the first. Assign the result back to the column: in pandas 3, `fillna(..., inplace=True)` on a single column no longer changes the DataFrame (pandas docs, Copy-on-Write).

Extra: Two slips are easy to make here.

- **Imputing before splitting.** Calling `fillna` with the mode of the whole table, `df["GarageQual"].mode()[0]`, learns the mode from the test rows too. On this data the mode is TA either way, but in general this is data leakage.
- **Transforming the wrong set.** `X_test = imp.transform(X_train)` fills the training set again and stores it as the test set. The line must read `imp.transform(X_test)`.

6. Missing category imputation on real data

Key point: With the "Missing" category, the 547 gaps in `FireplaceQu` become the largest category, while Gd, TA and the rest keep their exact counts.

Filling the gaps with "Missing" adds one bar to each column (Figure 6). In `GarageQual` it is a small bar of 64 rows (5.5%). In `FireplaceQu` it becomes the largest category, with 547 rows (46.8%).

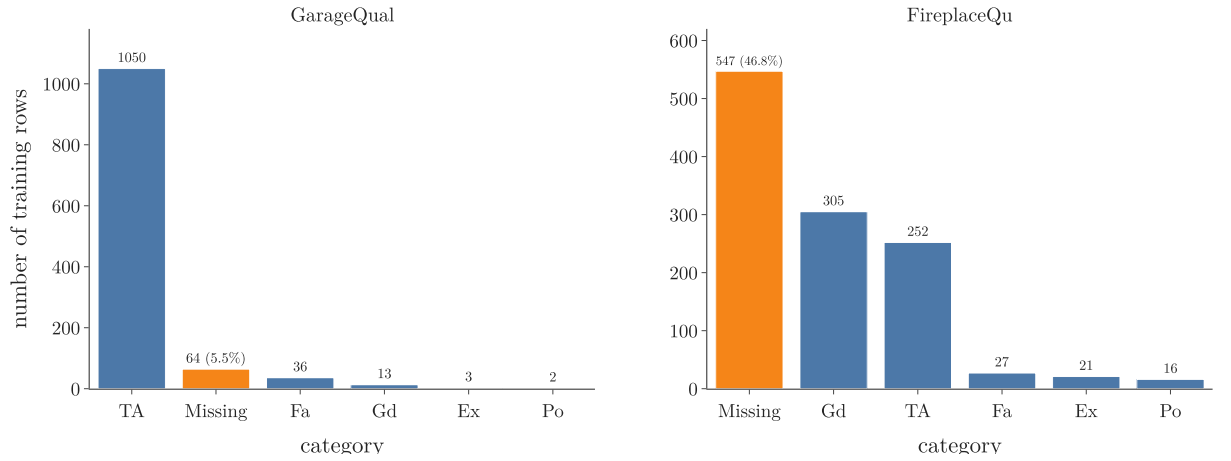


Figure 6: Category counts in the training set after adding the category "Missing" (orange); every other count is unchanged

Every original category keeps exactly the count it had: 305 Gd, 252 TA and so on. Nothing is inflated, and the model can still learn that houses in the "Missing" group sold for less.

Python: The "Missing" category with `SimpleImputer`.

```
imp = SimpleImputer(strategy="constant",
                    fill_value="Missing")
X_train = imp.fit_transform(X_train)
X_test = imp.transform(X_test)
imp.statistics_           # ['Missing', 'Missing']
```

`strategy="constant"` fills every gap with `fill_value`. The constant strategy learns nothing from the data, so `fit` only records the fill value. In pandas, the same is `X_train[col].fillna("Missing")`.

7. Choosing between the two

Key point: Few gaps and one dominant category: **mode** (G-1251). Many gaps, no dominant category, or gaps with a reason: "Missing".

	Mode imputation	"Missing" category
Fills a gap with	the most frequent category	the word "Missing"
Categories after	the same	one more
Changes existing shares	yes: the mode grows	no
Use when	MCAR, about 5% missing or less, one category dominates	many values missing, or not MCAR
`SimpleImputer`	`strategy="most_frequent"`	`strategy="constant", fill_value="Missing"`
In this data	fine for `GarageQual`	better for `FireplaceQu`

Figure 7 turns the table into three questions asked in order. Watch where each column leaves the path: `FireplaceQu` fails the first two questions, while `GarageQual` fails only the third, with too few gaps for the choice to matter much.

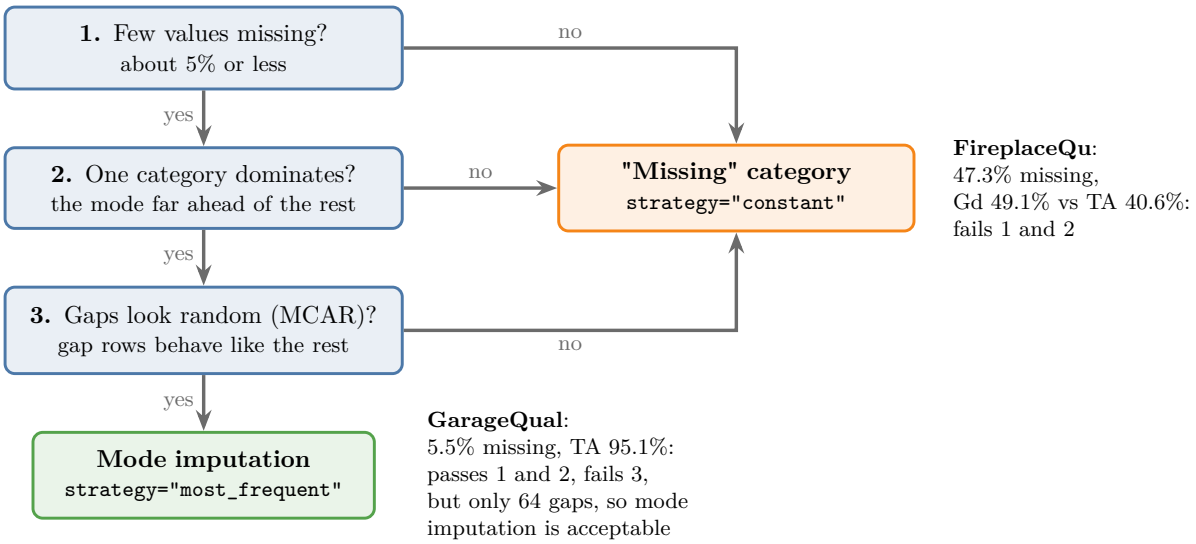


Figure 7: Choosing the imputer: three questions in order, with the two house-price columns placed where they end

> **Extra:** What the gaps mean in this data.
> In this dataset, the gaps are not lost values at all. The dataset's own description says that an empty `FireplaceQu` means "no fireplace" and an empty `GarageQual` means "no garage" (De Cock 2011). The data confirms it: all 690 houses with a gap in `FireplaceQu` have 0 fireplaces, and all 81 with a gap in `GarageQual` also have no garage type.
> So the value is missing because the thing itself does not exist: no fireplace, or no garage (De Cock 2011). For both columns, a category such as "Missing" (or better, "None") describes the truth; guessing TA or Gd invents a garage or a fireplace.
> This also explains why the houses with a gap sold for less in Figure 5: a house with no garage or no fireplace lacks something buyers pay for. In a study of more than 28,800 home sales around Philadelphia, a garage added about 13% to the selling price and each fireplace about 12% (Sirmans and Macpherson 2003).
> Before imputing, it is worth reading the data's description to see whether "missing" has a meaning.

8. Summary

- A categorical column has no mean or median, so its gaps are filled with the **mode** (G-1251) or with a new category.
- **Mode imputation** fills every gap with the most frequent category. Mode imputation suits **MCAR** (G-1192) data with few gaps (about 5% or less) and one dominant category, because if the categories are nearly tied, guessing the mode for every gap is often wrong; the method is easy to deploy but inflates the mode.
- **Missing category imputation** fills every gap with "Missing". The "Missing" category suits **features** (G-772) with many gaps or gaps that are not random, because the model can then learn whether a missing value itself matters. Existing categories keep their counts.
- After imputing, compare each category's share before and after, because a large jump shows the imputation changed the column. On the house data, TA in `GarageQual` moved from 95.1% to 95.4%, but Gd in `FireplaceQu` jumped from 49.1% to 72.9%.
- Also compare the target for the most frequent category against the **observations** (G-1374) with gaps, because a clear difference shows the gaps are not MCAR: here the gaps marked cheaper houses, because a gap means the house has no garage or no fireplace.
- Split first, then fit `SimpleImputer` on the training set and transform both sets with it, so the mode is learned from the training set alone and no test information leaks into training.
- So a gap in a categorical feature gets the mode when few values are missing and one category dominates, and a "Missing" category otherwise.

9. Sources

Built from

- CampusX, "Handling Missing Categorical Data | Simple Imputer | Most Frequent Imputation | Missing Category Imp", YouTube, https://www.youtube.com/watch?v=l_Wip8bEDFQ

Other references

- De Cock, D. (2011). Ames, Iowa: Alternative to the Boston Housing Data as an End of Semester Regression Project. *Journal of Statistics Education* 19(3); its data documentation file.
- Sirmans, G. S. and Macpherson, D. A. (2003). *The Value of Housing Characteristics.* Research brief, National Association of Realtors, National Center for Real Estate Research, December 2003 (Findings: garage about +13%, each fireplace about +12%; 28,800 sales, Philadelphia area, 1996–2003).
- pandas User Guide, *Copy-on-Write (CoW)*, section on chained assignment.

10. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Missing category imputation (G-1232)	Filling every gap in a categorical column with a new category, "Missing".
Most frequent value imputation (mode imputation) (G-1264)	Filling every gap in a column with its mode.
`strategy="constant"` (SimpleImputer) (G-147)	The `SimpleImputer` setting that fills every gap with `fill_value`.
`strategy="most_frequent"` (SimpleImputer) (G-148)	The `SimpleImputer` setting for mode imputation: `fit` learns each column's most frequent value and `transform` fills the gaps with it.
Category share (G-355)	The rows in one category divided by the rows that have a value.
Mode (G-1251)	The value that appears most often in the data, so it sits under the highest peak of a histogram or a distribution curve; it gives a centre for data that are not numbers, and a belief with several peaks has several modes.
Feature (G-772)	One piece of information about each example that a model uses (e.g. a student's CGPA).
Target (G-1949)	The output column we predict, such as the class; also called the label.
Observation (G-1374)	One record of a dataset: one row of the data table.
Univariate imputation (G-2051)	Filling the gaps in a column (imputation) using only that column's other values.
SimpleImputer (G-1809)	scikit-learn's class that fills missing values, by default with the column's mean.
Random sample imputation (G-1616)	Filling each gap with a value drawn at random from the column's known values.
MCAR (G-1192)	Missing completely at random: the gaps have no relation to any value in the data.
Arbitrary value imputation (G-208)	Filling every gap with one fixed value that never occurs, such as 99 or -1.
MAR (G-1158)	Missing at random: the gaps depend on another, recorded column.
MNAR (G-1248)	Missing not at random: the gaps depend on the missing value itself.
One-hot encoding (G-1379)	Replacing a nominal column by one 0/1 column per category, with a single 1 in each row; also used to represent words.

Term	Meaning
Missing indicator (G-1233)	A 0/1 (True/False) column recording whether a value was missing, added next to the imputed column so the model can learn whether being missing itself matters.
Data leakage (G-535)	Information from the test rows reaching the model during training, for example when a scaling or feature-selection step is fitted on all the data before the split; the test scores then come out higher than the model will really do on new data.
Kernel density estimate (KDE) (G-1005)	A smooth curve that estimates a column's PDF from its values, built by adding a kernel centred on every data point; a KDE plot draws it.
`statistics_` (G-145)	The fill values a fitted `SimpleImputer` has learned, one per column.